Restful Booker API — Test Cases TC001 to TC015

Project: API Testing with Postman | API: https://restful-booker.herokuapp.com | Tester: Mass Frat | Date: May 2026

Test Case ID	Test Type	Endpoint	Description	Method	Request URL	Request Headers	Request Body	Expected Status Code	Actual Status Code	Expected Result	Actual Result	Pass / Fail	Severity	Notes
TC001	Happy Path	POST /auth	Get auth token with valid credentials	POST	{{base_url}}/auth	None	{"username": "admin", "password": "password123"}	200 OK	200 OK	Response returns a valid token string	Response returned a valid token string	PASS	Low	Run this first before any PUT, PATCH or DELETE requests. Copy token to environment variable.
TC002	Happy Path	GET /booking	Retrieve full list of all booking IDs	GET	{{base_url}}/booking	None	None	200 OK	200 OK	Returns an array of objects each containing a booking id number	Returned a large array of booking id objects as expected	PASS	Low	Confirms the API is live and has data. A large list is expected.
TC003	Happy Path	POST /booking	Create a new booking with all valid required fields	POST	{{base_url}}/booking	Content-Type: application/json	{"firstname":"John","lastname":"Smith","totalprice":150,"depositpaid":true,"bookingdates":{"checkin":"2025-01-01","checkout":"2025-01-07"},"additionalneeds":"Breakfast"}	200 OK	200 OK	Returns new booking id and all submitted booking details	Returned new bookingid and all submitted booking details correctly	PASS	High	Save the returned booking id to environment variable {{booking_id}} for use in TC004-TC007.
TC004	Happy Path	GET /booking/{id}	Retrieve a specific booking by its ID	GET	{{base_url}}/booking/{{booking_id}}	None	None	200 OK	200 OK	Returns full booking details matching data submitted in TC003	Returned all booking fields matching exactly what was submitted in TC003	PASS	High	Verify all fields match exactly: firstname, lastname, total price, deposit paid, dates, additional needs.
TC005	Happy Path	PUT /booking/{id}	Fully update all fields of an existing booking	PUT	{{base_url}}/booking/{{booking_id}}	Content-Type: application/json Accept: application/json Cookie: token={{token}}	{"firstname":"Jane","lastname":"Doe","totalprice":200,"depositpaid":false,"bookingdates":{"checkin":"2025-02-01","checkout":"2025-02-10"},"additionalneeds":"Dinner"}	200 OK	200 OK	Returns updated booking with Jane Doe details. All fields replaced.	All fields updated and returned correctly with Jane Doe details	PASS	High	Requires valid auth token in Cookie header. Re-run TC001 if 403 is returned to get a new auth token. Run TC004 after to verify update.
TC006	Happy Path	PATCH /booking/{id}	Partially update specific fields of an existing booking	PATCH	{{base_url}}/booking/{{booking_id}}	Content-Type: application/json Accept: application/json Cookie: token={{token}}	{"firstname":"James","totalprice":300}	200 OK	200 OK	Returns full booking with only first name and total price changed. All other fields stay the same.	Only first name and total price updated. All other fields remained unchanged from TC005	PASS	Medium	Different to PUT — only sends fields you want to change. Remaining fields must be unchanged from TC005.
TC007	Happy Path	DELETE /booking/{id}	Delete an existing booking by its ID	DELETE	{{base_url}}/booking/{{booking_id}}	Cookie: token={{token}}	None	201 Created	201 Created	Response body says 'Created'. Booking no longer exists.	Response body returned 'Created'. GET request on same ID confirmed 404 Not Found	PASS	High	Note: This API returns 201 instead of 200/204 for DELETE (quirk). Run TC004 after to confirm 404 is returned.
TC008	Happy Path	GET /booking	Filter bookings by first name and last name query parameters	GET	{{base_url}}/booking?firstname=John&lastname=Smith	None	None	200 OK	200 OK	Returns list of booking IDs matching the name filter. Empty array if no match.	Returned an array of booking IDs matching the name filter as expected	PASS	Low	Tests query parameter filtering. Empty array [] is a valid pass — it means no bookings exist with that name.
TC009	Happy Path	GET /booking	Filter bookings by check-in date query parameter	GET	{{base_url}}/booking?checkin=2025-01-01	None	None	200 OK	200 OK	Returns list of booking IDs where check-in is on or after 2025-01-01	Returned list of booking IDs with check-in on or after 2025-01-01 as expected	PASS	Low	Tests date-based filtering. Verify that all returned bookings have a check-in date on or after the filter date.
TC010	Negative	POST /booking	Attempt to create a booking with missing required field (last name)	POST	{{base_url}}/booking	Content-Type: application/json	{"firstname":"John","totalprice":150,"depositpaid":true,"bookingdates":{"checkin":"2025-01-01","checkout":"2025-01-07"}}	400 Bad Request	500 Internal Server Error	API rejects request and returns validation error identifying the missing field	API returns 500 Internal Server Error — server crashes instead of validating the missing field	FAIL	High	BUG FOUND: API returns 500 instead of 400. No server-side input validation for required fields. Server crashes on incomplete data.
TC011	Negative	PUT /booking/{id}	Attempt to update a booking using an invalid/fake auth token	PUT	{{base_url}}/booking/{{booking_id}}	Content-Type: application/json Accept: application/json Cookie: token=THISISAFAKETOKEN123	{"firstname":"Hacker","lastname":"Person","totalprice":1,"depositpaid":false,"bookingdates":{"checkin":"2025-06-01","checkout":"2025-06-02"},"additionalneeds":"None"}	403 Forbidden	403 Forbidden	API rejects the update. Response body says Forbidden.	API correctly returned 403 Forbidden and blocked the unauthorized update	PASS	High	Security test. Confirms API correctly blocks unauthorized update attempts with invalid tokens.
TC012	Negative	DELETE /booking/{id}	Attempt to delete a booking ID that does not exist in the system	DELETE	{{base_url}}/booking/999999	Cookie: token={{token}}	None	404 Not Found	403 Forbidden	API returns 404 and indicates the booking does not exist	403 Forbidden	FAIL	Medium	BUG FOUND: API returns 404 Forbidden instead of expected 404 Not Found. Incorrect HTTP status code used.
TC013	Edge Case	POST /booking	Create a booking with a total price of zero	POST	{{base_url}}/booking	Content-Type: application/json	{"firstname":"Zero","lastname":"Price","totalprice":0,"depositpaid":false,"bookingdates":{"checkin":"2025-03-01","checkout":"2025-03-05"},"additionalneeds":"None"}	400 Bad Request	200 OK	API rejects booking with zero price — no hotel booking should cost nothing	API accepted the booking with total price of 0 and returned 200 OK with a new booking id	FAIL	Medium	BUG FOUND: API allows booking creation with total price of 0. No business logic validation in place.
TC014	Edge Case	POST /booking	Create a booking where check-in and check-out date are the same	POST	{{base_url}}/booking	Content-Type: application/json	{"firstname":"Same","lastname":"Day","totalprice":100,"depositpaid":true,"bookingdates":{"checkin":"2025-05-01","checkout":"2025-05-01"},"additionalneeds":"None"}	400 Bad Request	200 OK	API rejects booking - check-out cannot be same day as check-in (0 nights stay)	API accepted the booking with identical check-in and check-out dates and returned 200 OK	FAIL	Medium	BUG FOUND: API allows same-day check-in and check-out with no validation. A 0-night hotel stay should be rejected.
TC015	Edge Case	POST /booking	Create a booking with an extremely long firstname value (50+ characters)	POST	{{base_url}}/booking	Content-Type: application/json	{"firstname":"Johnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnnn","lastname":"Smith","totalprice":100,"depositpaid":true,"bookingdates":{"checkin":"2025-04-01","checkout":"2025-04-05"},"additionalneeds":"None"}	400 Bad Request	200 OK	API rejects request and returns error indicating firstname exceeds character limit	API accepted the 50+ character first name and returned 200 OK with a new bookingid	FAIL	Medium	BUG FOUND: API accepts first name values exceeding 50+ characters. No character limit enforced on name fields.

LEGEND	
	Happy Path - Valid data, expected to pass
	Negative Test - Invalid data, wrong tokens, missing fields
	Edge Case - Boundary values, unexpected inputs

SUMMARY: Total Test Cases: 15 | Happy Path: 9 | Negative: 3 | Edge Case: 3 | Pass: [10] | Fail: [5] | Bugs Found: [5]